

Università degli Studi dell'Insubria
Laurea Triennale in Informatica

Corso: **Interfacce Uomo-Macchina** – a.a. 2025/26

Docente: Prof.ssa Silvia Corchs

Documentazione Tecnica

Pipeline di Feature Extraction e Machine Learning EEG
per la Classificazione di Decisioni
nell'Iowa Gambling Task

Dataset di riferimento:

Chávez-Sánchez M., Torres-Ramos S., Román-Godínez I., Salido-Ruiz R.A.
*Behavioral and electroencephalographic dataset simultaneously acquired
during the Iowa Gambling Task*
Scientific Data, 13:359, 2026
DOI: [10.1038/s41597-026-06662-0](https://doi.org/10.1038/s41597-026-06662-0)

Dataset disponibile su:

[Mendeley Data – 2pw2m39yct](#), versione 2

Indice

1	Introduzione e Obiettivi	3
1.1	Contesto del progetto	3
1.2	Obiettivo della classificazione	3
1.3	Struttura della documentazione	3
2	Architettura della Pipeline	4
2.1	Panoramica generale	4
2.2	Dipendenze software	4
3	Feature Extraction – PSD (Welch)	5
3.1	Motivazione della scelta PSD	5
3.2	Algoritmo di Welch	5
3.3	Bande di frequenza	6
3.4	Implementazione: <code>extract_psd_features()</code>	6
3.5	Struttura della feature matrix	7
4	Definizione dei Modelli di Machine Learning	8
4.1	Logistic Regression	8
4.2	Support Vector Machine (SVM, kernel RBF)	8
4.3	Random Forest	9
4.4	Pipeline sklearn e prevenzione del data leakage	9
5	Validazione: Leave-One-Subject-Out (LOSO)	10
5.1	Motivazione della LOSO	10
5.2	Parallelizzazione dei fold	10
5.3	GroupKFold come alternativa	11
6	Metriche di Valutazione	12
6.1	Metriche calcolate per fold	12
6.2	Aggregazione e tabella riepilogativa	12
7	Visualizzazioni	13
7.1	Confusion Matrix aggregata	13
7.2	ROC Curve media con deviazione standard	13
7.3	Feature Importance	13
8	Comandi di Esecuzione della Pipeline	15
8.1	Modalità LOSO completa (raccomandata per risultati pubblicabili)	15
8.2	Modalità GroupKFold (test rapido)	16
8.3	Modalità con dati sintetici	18
8.4	Riepilogo argomenti da riga di comando	18
8.5	File di output prodotti	18
9	Risultati Ottenuti e Confronto con la Letteratura	19
9.1	Statistiche del dataset preprocessato	19
9.2	Risultati LOSO (media $\pm$ std su 55 fold)	19
9.3	Confronto con la letteratura EEG-IGT	20

10 Scelte Implementative e Motivazioni	21
10.1 PSD invece di analisi ERP	21
10.2 Classificazione binaria (A/B vs C/D)	21
10.3 Gestione sbilanciamento: <code>class_weight='balanced'</code>	21
10.4 Parallelizzazione con backend <code>loky</code>	21
10.5 Finestra pre-decisionale $[-2s, 0s]$	21
11 Conclusioni e Sviluppi Futuri	23
11.1 Riepilogo	23
11.2 Sviluppi futuri (prossime fasi)	23

1 Introduzione e Obiettivi

1.1 Contesto del progetto

Il presente documento descrive nel dettaglio la pipeline di feature extraction e machine learning (ML) implementata nella Parte II del progetto d'esame del corso di Interfacce Uomo-Macchina. La pipeline riceve in input le epoche EEG già preprocessate dalla Parte I (pipeline di preprocessing descritta nella documentazione tecnica precedente) e produce come output modelli di classificazione addestrati e valutati con metodologia cross-subject rigorosa.

La Parte I ha prodotto, per ogni soggetto, un tensore di epoche EEG pulite di shape (200, 21, 513), corrispondente a 200 trial, 21 canali e 513 campioni ($2\text{ s} \times 256\text{ Hz} + 1$) per la finestra temporale $[-2\text{ s}, 0\text{ s}]$ prima del momento decisionale. Il preprocessing ha incluso: caricamento e conversione in MNE `RawArray`, re-referencing CAR, filtraggio bandpass FIR 0.5–70 Hz, filtraggio notch a 50 Hz, rimozione artefatti via ICA FastICA ($K = 15$ componenti), normalizzazione z-score per canale.

1.2 Obiettivo della classificazione

Task di classificazione

Input: epoche EEG di shape (n_epochs, 21, 513) – finestra $[-2\text{ s}, 0\text{ s}]$ pre-decisione.

Output: classe binaria

- **Classe 0** – Decisione *svantaggiosa* (deck A o B)
- **Classe 1** – Decisione *vantaggiosa* (deck C o D)

1.3 Struttura della documentazione

Il documento è organizzato come segue:

1. **Sezione 2:** architettura generale della pipeline e dipendenze software
2. **Sezione 3:** feature extraction tramite Power Spectral Density (metodo di Welch)
3. **Sezione 4:** definizione e configurazione dei modelli ML
4. **Sezione 5:** strategia di validazione Leave-One-Subject-Out (LOSO)
5. **Sezione 6:** metriche di valutazione e tabella riepilogativa
6. **Sezione 7:** visualizzazioni (Confusion Matrix, ROC Curve, Feature Importance)
7. **Sezione 8:** comandi di esecuzione della pipeline
8. **Sezione 9:** risultati ottenuti e confronto con la letteratura
9. **Sezione 10:** scelte implementative e motivazioni
10. **Sezione 11:** conclusioni e sviluppi futuri

2 Architettura della Pipeline

2.1 Panoramica generale

La pipeline ML è strutturata in moduli logicamente separati e sequenziali, ciascuno implementato come funzione autonoma e testabile nel file `src/eeg_ml_pipeline.py`:

Panoramica della pipeline

Input: tensori `.npy` per soggetto (200, 21, 513) + labels (200,)

Step 1: Caricamento dati multi-soggetto – `load_preprocessed_data()`

Step 2: Feature Extraction PSD (Welch) – `build_feature_matrix()` → X shape (11000, 105)

Step 3: Inizializzazione modelli ML – `get_models()` → {LR, SVM-RBF, RF}

Step 4: Cross-validation LOSO parallelizzata – `run_cross_validation()`

Step 5: Aggregazione metriche (media $\pm$ std) – `compute_summary_table()`

Step 6: Visualizzazioni – CM, ROC, Feature Importance, Boxplot

Step 7: Salvataggio – `results_summary.csv` + modelli `.joblib`

Output: directory `ml_results/` con CSV, PNG e modelli serializzati

2.2 Dipendenze software

Tabella 1: Dipendenze software della pipeline ML

Libreria	Versione min.	Utilizzo
<code>numpy</code>	≥ 1.24	Array multidimensionali, operazioni vettoriali
<code>scipy</code>	≥ 1.10	Calcolo PSD (<code>scipy.signal.welch</code>)
<code>pandas</code>	≥ 2.0	Tabelle risultati, salvataggio CSV
<code>scikit-learn</code>	≥ 1.3	Modelli ML, Pipeline, LOSO, metriche
<code>joblib</code>	≥ 1.3	Parallelizzazione fold LOSO
<code>matplotlib</code>	≥ 3.7	Generazione grafici (ROC, CM, boxplot)
<code>seaborn</code>	≥ 0.12	Heatmap Confusion Matrix
<code>mne</code>	≥ 1.6	Compatibilità formato epoche (Parte I)

3 Feature Extraction – PSD (Welch)

3.1 Motivazione della scelta PSD

La Power Spectral Density (PSD) è la feature EEG più consolidata per la classificazione di stati cognitivi. Rispetto all'analisi ERP adottata nella validazione del paper originale [1], la PSD è preferibile per la classificazione *trial-by-trial* per le seguenti ragioni:

- **Non richiede averaging su multipli trial:** le feature ERP emergono affidabilmente solo dopo mediazione su decine o centinaia di trial. Con ~ 200 trial per soggetto, le feature ERP di singolo trial sono troppo rumorose per un classificatore generalizzabile.
- **Robustezza al rumore:** il metodo di Welch riduce la varianza della stima PSD rispetto alla FFT diretta, producendo feature più stabili su finestre brevi (2 s).
- **Coerenza con la letteratura EEG-IGT:** Aram et al. [6] e Balconi & Angioletti [5] identificano nella potenza spettrale per banda (theta frontale, alpha occipitale) i correlati neurali più informativi per la distinzione vantaggioso/svantaggioso nell'IGT.
- **Corrispondenza con gli ERP del paper:** la potenza theta in Fz (4–8 Hz) è il correlato spettrale della componente FRN (Feedback-Related Negativity) identificata dal paper in Fig. 9a.

3.2 Algoritmo di Welch

Il metodo di Welch stima la PSD dividendo il segnale in segmenti sovrapposti, calcolando il periodogramma modificato per ciascuno e mediando:

Stima PSD – Metodo di Welch

$$\hat{P}(f) = \frac{1}{K} \sum_{k=1}^K \frac{|\text{DFT}[w \cdot x_k](f)|^2}{f_s \cdot \sum_n w^2(n)}$$

dove K = numero segmenti sovrapposti, w = finestra di Hamming, $f_s = 256$ Hz.

La potenza media nella banda $[f_{\text{low}}, f_{\text{high}}]$ è:

Potenza di banda

$$P_{\text{banda}} = \frac{1}{|\mathcal{F}|} \sum_{f \in \mathcal{F}} \hat{P}(f), \quad \mathcal{F} = \{f : f_{\text{low}} \leq f < f_{\text{high}}\}$$

Parametri implementativi: $n_{\text{perseg}} = \min(256, n_{\text{samples}}) = 256$ campioni (= 1 s a 256 Hz), finestra di Hamming, overlap 50% (default `scipy.signal.welch`). La risoluzione frequenziale risultante è $\Delta f = f_s / n_{\text{perseg}} = 1$ Hz.

3.3 Bande di frequenza

Vengono estratte cinque bande di frequenza standard, coerenti con le bande neurofisiologicamente rilevanti descritte nella documentazione del preprocessing (Tabella 5 della Parte I):

Tabella 2: Bande di frequenza estratte e correlati neurali nell'IGT

Banda	Range (Hz)	Elettrodi chiave	Rilevanza IGT
Delta (δ)	0.5–4	Frontale, Parietale	Processi decisionali lenti, attenzione sostenuta
Theta (θ)	4–8	Fz, F3, F4, F7, F8	Working memory, controllo cognitivo, FRN
Alpha (α)	8–13	O1, O2, P3, P4	Inibizione cognitiva, resting state, P300
Beta (β)	13–30	Frontale, Centrale	Elaborazione sensoriale, preparazione motoria
Gamma (γ)	30–45	Distribuito	Binding cognitivo, integrazione (N400)

Nota sul limite superiore della banda gamma

Il paper usa un taglio alto a 70 Hz per il filtraggio. La feature gamma viene estratta fino a 45 Hz (non 70 Hz) perché al di sopra di 45 Hz l'attività EMG muscolare residua domina sul segnale neurale, introducendo feature spurie. Il range 30–45 Hz è lo standard per la banda gamma nei task cognitivi.

3.4 Implementazione: `extract_psd_features()`

Listing 1: Funzione `extract_psd_features()`

```

1 def extract_psd_features(epoch, sfreq=256, freq_bands=
    FREQ_BANDS,
2                             nperseg=None):
3     """
4     Input:  epoch      -- np.ndarray (n_channels, n_samples)
5             sfreq      -- frequenza di campionamento (Hz)
6             freq_bands -- dict {banda: (low_hz, high_hz)}
7     Output: features -- np.ndarray (n_channels * n_bands,)
8     """
9     n_channels, n_samples = epoch.shape
10    nperseg = min(256, n_samples) # finestra 1s
11
12    features = []
13    for ch_idx in range(n_channels):
14        freqs, psd = welch(epoch[ch_idx], fs=sfreq,
15                           nperseg=nperseg, window='hann')
16        for band_name, (low, high) in freq_bands.items():
17            mask = (freqs >= low) & (freqs < high)

```

```

18         band_power = np.mean(psd[mask]) if mask.sum() > 0
19     else 0.0
20         features.append(band_power)
21     return np.array(features, dtype=np.float32)

```

3.5 Struttura della feature matrix

Il vettore feature per una singola epoca ha struttura:

Struttura del vettore feature per epoca

$$\mathbf{x} = [\underbrace{x_{1\delta}, x_{1\theta}, x_{1\alpha}, x_{1\beta}, x_{1\gamma}}_{\text{CH}_1}, \underbrace{x_{2\delta}, \dots, x_{2\gamma}}_{\text{CH}_2}, \dots, \underbrace{x_{21\delta}, \dots, x_{21\gamma}}_{\text{CH}_{21}}]$$

dimensione: 21 canali $\times$ 5 bande = 105 feature per epoca

La feature matrix completa è:

$$\mathbf{X} \in \mathbb{R}^{n_{\text{epochs}} \times 105}, \quad n_{\text{epochs}} = 55 \text{ soggetti} \times 200 \text{ trial} = 11.000$$

4 Definizione dei Modelli di Machine Learning

Vengono implementati tre classificatori con caratteristiche complementari, ciascuno incapsulato in una `sklearn.Pipeline` che include obbligatoriamente uno `StandardScaler` come primo step. Tutti i modelli usano `class_weight='balanced'` (discusso in Sezione 10.3).

4.1 Logistic Regression

La Logistic Regression è il baseline lineare della pipeline. Modella la probabilità di classe come:

Logistic Regression

$$P(y = 1 \mid \mathbf{x}) = \sigma(\beta_0 + \boldsymbol{\beta}^\top \mathbf{x}), \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

con regolarizzazione L2: $\mathcal{L} = -\sum_i \log P(y_i \mid \mathbf{x}_i) + \frac{1}{2C} \|\boldsymbol{\beta}\|^2$

Parametri: $C = 1.0$, `solver=lbfgs`, `max_iter=1000`. Il vettore $\boldsymbol{\beta}$ fornisce interpretabilità diretta: $\beta_j > 0$ indica associazione con la classe vantaggiosa (C/D), $\beta_j < 0$ con la classe svantaggiosa (A/B).

Listing 2: Configurazione Logistic Regression Pipeline

```

1 Pipeline([
2     ('scaler', StandardScaler()),
3     ('clf', LogisticRegression(
4         C=1.0,
5         solver='lbfgs',
6         max_iter=1000,
7         class_weight='balanced',
8         random_state=42
9     ))
10 ])
```

4.2 Support Vector Machine (SVM, kernel RBF)

L'SVM con kernel RBF proietta i dati in uno spazio ad alta dimensione per trovare un iperpiano separatore non lineare:

SVM – Kernel RBF

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2), \quad \gamma = \frac{1}{d \cdot \text{Var}(\mathbf{X})} \quad (\text{scale})$$

Parametri: $C = 1.0$, `probability=True` (Platt scaling per AUC).

Nota computazionale – SVM

La complessità di training dell'SVM con kernel RBF è $\mathcal{O}(n^2)$ rispetto al numero di campioni. Con ~ 9.800 campioni di training per fold LOSO, il training di ogni fold richiede $\sim 10\text{--}30$ secondi su CPU consumer. La parallelizzazione LOSO (Sezione 5) mitiga significativamente questo costo.

4.3 Random Forest

Il Random Forest è un ensemble di $N = 200$ alberi decisionali addestrati su sottocampioni casuali dei dati e delle feature (bagging + random subspace):

Random Forest – Predizione aggregata

$$\hat{y} = \text{mode}\{T_1(\mathbf{x}), T_2(\mathbf{x}), \dots, T_{200}(\mathbf{x})\}$$

Parametri: `n_estimators=200`, `min_samples_split=5`,
`min_samples_leaf=2`, `n_jobs=-1`.

Il Random Forest fornisce come prodotto secondario la stima dell'importanza delle feature tramite *mean decrease impurity* (Gini), permettendo di identificare quali bande EEG e quali elettrodi sono maggiormente informativi.

4.4 Pipeline sklearn e prevenzione del data leakage

Un aspetto metodologicamente critico è la corretta gestione dello `StandardScaler` durante la cross-validation. Lo scaler deve essere fittato esclusivamente sui dati di training di ogni fold, mai sui dati di test:

Listing 3: Approccio corretto – StandardScaler dentro Pipeline

```

1 # SBAGLIATO: scaler vede tutto il dataset (anche il test)
2 scaler.fit(X_all)
3 X_scaled = scaler.transform(X_all) # data leakage!
4
5 # CORRETTO: scaler dentro Pipeline, fittato solo su train
6 pipeline = Pipeline([
7     ('scaler', StandardScaler()),
8     ('clf', RandomForestClassifier()),
9 ])
10 pipeline.fit(X_train, y_train) # scaler fittato SOLO su
    X_train

```

5 Validazione: Leave-One-Subject-Out (LOSO)

5.1 Motivazione della LOSO

Leave-One-Subject-Out (LOSO)

I dati EEG sono strettamente soggetto-dipendenti: ogni individuo ha pattern spettrali unici legati alla morfologia del cranio, all'impedenza degli elettrodi e all'attività basale. Una divisione random train/test che mescola trial dello stesso soggetto produce stime ottimisticamente distorte (*data leakage* inter-soggetto).

La LOSO garantisce che ogni fold utilizzi un soggetto diverso come test set, stimando la capacità di generalizzazione a nuovi individui mai visti durante il training. Con 55 soggetti, la LOSO produce **55 fold indipendenti**.

La LOSO è implementata tramite `LeaveOneGroupOut` di scikit-learn, con `groups=subject_ids`:

Listing 4: LOSO cross-validation con scikit-learn

```

1 from sklearn.model_selection import LeaveOneGroupOut
2
3 cv = LeaveOneGroupOut()
4 for train_idx, test_idx in cv.split(X, y, groups=subject_ids)
5     :
6     # GARANZIA: soggetto test NON in train
7     X_train, X_test = X[train_idx], X[test_idx]
8     y_train, y_test = y[train_idx], y[test_idx]
9     pipeline.fit(X_train, y_train)      # scaler fittato su
10    train
11    y_pred = pipeline.predict(X_test)
12    y_proba = pipeline.predict_proba(X_test)[: , 1]
```

5.2 Parallelizzazione dei fold

Con $55 \text{ fold} \times 3 \text{ modelli} \times \sim 9.800 \text{ campioni di training per fold}$, l'esecuzione sequenziale richiede tempi proibitivi (stimati > 30 minuti su hardware consumer). I fold LOSO sono matematicamente indipendenti, rendendo la parallelizzazione triviale e senza impatto sui risultati:

Listing 5: Parallelizzazione LOSO con joblib

```

1 from joblib import Parallel, delayed
2 from sklearn.base import clone
3
4 fold_results = Parallel(n_jobs=-1, backend='loky')(
5     delayed(_run_single_fold)(
6         fold_idx, train_idx, test_idx,
7         X, y, subject_ids,
8         models,          # clonati internamente via sklearn.base
9         .clone()
10        total_folds
11    )
12)
```

```
11     for fold_idx, (train_idx, test_idx) in enumerate(  
12         all_splits)  
    )
```

Nota implementativa – `clone()` di sklearn

Ogni processo worker crea copie fresche delle pipeline tramite `sklearn.base.clone()`. Questo garantisce:

- assenza di race condition tra processi (ogni worker ha il suo stato)
- ogni fold parte da un modello non ancora addestrato (correttezza statistica)
- il `random_state=42` globale garantisce riproducibilità

5.3 GroupKFold come alternativa

In alternativa alla LOSO, è disponibile la `GroupKFold` con k configurabile. Con $k = 5$, il dataset viene diviso in 5 fold mantenendo la separazione per soggetto. La `GroupKFold` è computazionalmente più veloce (k fold invece di 55) ma produce stime meno accurate della performance cross-subject. È raccomandata per test rapidi della pipeline prima di avviare la LOSO completa.

6 Metriche di Valutazione

6.1 Metriche calcolate per fold

Per ogni fold e per ogni modello vengono calcolate cinque metriche (classe positiva = classe 1, decisione vantaggiosa C/D):

Tabella 3: Metriche di valutazione calcolate per fold

Metrica	Formula	Interpretazione
Accuracy	$(TP + TN)/(TP + TN + FP + FN)$	Proporzione predizioni corrette totali
Precision	$TP/(TP + FP)$	Tra le predizioni C/D, quante sono corrette
Recall	$TP/(TP + FN)$	Tra i veri C/D, quanti vengono identificati
F1-Score	$2 \cdot P \cdot R / (P + R)$	Media armonica Precision e Recall
ROC-AUC	$\int_0^1 \text{TPR} d(\text{FPR})$	Area sotto la curva ROC

La **ROC-AUC** è la metrica principale: è robusta allo sbilanciamento delle classi e misura il potere discriminante del modello indipendentemente dalla soglia di decisione. $\text{AUC} = 0.5$ corrisponde al classificatore casuale; $\text{AUC} = 1.0$ alla classificazione perfetta.

6.2 Aggregazione e tabella riepilogativa

Al termine della LOSO, le metriche dei 55 fold vengono aggregate:

Aggregazione metriche LOSO

$$\bar{\mu}_{\text{metric}} = \frac{1}{55} \sum_{i=1}^{55} m_i, \quad \sigma_{\text{metric}} = \sqrt{\frac{1}{55} \sum_{i=1}^{55} (m_i - \bar{\mu})^2}$$

I valori NaN (fold con una sola classe nel test) vengono esclusi dall'aggregazione.

I risultati vengono salvati in `results_summary.csv` con colonne `{metrica}_mean` e `{metrica}_std` per ogni modello.

7 Visualizzazioni

7.1 Confusion Matrix aggregata

La CM viene costruita aggregando le predizioni di tutti i fold LOSO su tutti gli 11.000 trial, poi normalizzata per riga (normalizzazione per classe reale):

Confusion Matrix normalizzata

$$\widetilde{CM}_{ij} = \frac{CM_{ij}}{\sum_j CM_{ij}}$$

La diagonale principale corrisponde al recall per classe; gli elementi off-diagonali agli errori di classificazione.

La CM è generata per tutti e tre i modelli in un unico plot (1×3 subplots) tramite `seaborn.heatmap` con colormap Blues. Per ogni cella vengono riportati sia il valore normalizzato (proporzione) sia il conteggio assoluto (`n=...`).

7.2 ROC Curve media con deviazione standard

La curva ROC media viene costruita interpolando le TPR di ogni fold su una griglia comune di 200 punti in $[0, 1]$ per FPR, poi calcolando media e $\pm 1\sigma$:

Listing 6: Costruzione ROC curve media

```

1 mean_fpr = np.linspace(0, 1, 200)
2 tprs_interp = []
3 for fpr, tpr, auc_val in fold_rocs:
4     tprs_interp.append(np.interp(mean_fpr, fpr, tpr))
5     tprs_interp[-1][0] = 0.0      # forza partenza da (0,0)
6 mean_tpr = np.mean(tprs_interp, axis=0)
7 std_tpr = np.std(tprs_interp, axis=0)
8 # Banda +/- 1 std attorno alla curva media
9 ax.fill_between(mean_fpr,
10                 np.clip(mean_tpr - std_tpr, 0, 1),
11                 np.clip(mean_tpr + std_tpr, 0, 1), alpha=0.15)

```

7.3 Feature Importance

Il plot di feature importance visualizza le top-20 feature per importanza, calcolata sull'intero dataset a scopo esplicativo:

- **Random Forest:** importanza Gini (mean decrease impurity) con barre di errore = std tra i 200 alberi
- **Logistic Regression:** coefficienti β ordinati per $|\beta|$, con colore blu per $\beta_j > 0$ (associato C/D) e rosso per $\beta_j < 0$ (associato A/B)

Nota metodologica – Feature importance

Il plot di feature importance è puramente descrittivo e **non viene usato** per selezionare le feature prima della cross-validation (questo causerebbe data leakage). L'importanza viene stimata sull'intero dataset dopo la CV, esclusivamente per identificare i correlati neurali più informativi a scopo interpretativo.

8 Comandi di Esecuzione della Pipeline

La pipeline si avvia da terminale posizionandosi nella directory radice del progetto. I comandi sono identici su entrambi i sistemi operativi, con le sole differenze riportate di seguito per ogni sistema:

Differenze sintattiche tra sistemi operativi

Windows CMD: i percorsi usano il backslash (\), lo slash (/) per le opzioni e il carattere ^ per andare a capo su più righe.

macOS / Linux (Terminal): i percorsi usano lo slash (/), le opzioni sono identiche e il carattere \ serve per andare a capo.

Gli argomenti `-mode`, `-input`, `-output`, `-cv`, `-n_jobs`, `-n_splits` e `-no_save_models` sono identici su entrambi i sistemi.

Posizionarsi nella directory del progetto

Listing 7: Navigazione – Windows CMD

```
cd \percorso\verso\eeg_igt_pipeline
```

Listing 8: Navigazione – macOS / Linux Terminal

```
cd /percorso/verso/eeg_igt_pipeline
```

8.1 Modalità LOSO completa (raccomandata per risultati pubblicabili)

La LOSO è la strategia di validazione più rigorosa: produce 55 fold (uno per soggetto), garantisce l'assenza di data leakage inter-soggetto e stima la capacità di generalizzazione a nuovi individui.

LOSO – Tutti i core (più veloce)

Listing 9: Windows CMD

```
python src\eeg_ml_pipeline.py ^
    --mode real ^
    --input .\output ^
    --output .\ml_results_loso ^
    --cv loso
```

Listing 10: macOS / Linux Terminal

```
python3 src/eeg_ml_pipeline.py \
    --mode real \
    --input ./output \
    --output ./ml_results_loso \
    --cv loso
```


LOSO – Core limitati (per non bloccare il PC)

Listing 11: Windows CMD

```
python src\eeg_ml_pipeline.py ^
  --mode real ^
  --input .\output ^
  --output .\ml_results_loso ^
  --cv loso ^
  --n_jobs 4
```

Listing 12: macOS / Linux Terminal

```
python3 src/eeg_ml_pipeline.py \
  --mode real \
  --input ./output \
  --output ./ml_results_loso \
  --cv loso \
  --n_jobs 4
```

LOSO – Senza salvare i modelli (solo risultati)

Listing 13: Windows CMD

```
python src\eeg_ml_pipeline.py ^
  --mode real ^
  --input .\output ^
  --output .\ml_results_loso ^
  --cv loso ^
  --no_save_models
```

Listing 14: macOS / Linux Terminal

```
python3 src/eeg_ml_pipeline.py \
  --mode real \
  --input ./output \
  --output ./ml_results_loso \
  --cv loso \
  --no_save_models
```

8.2 Modalità GroupKFold (test rapido)

La GroupKFold con k fold è computazionalmente più veloce della LOSO pur mantenendo la separazione per soggetto. È raccomandata per verificare rapidamente il corretto funzionamento della pipeline prima di avviare la LOSO completa.

GroupKFold con $k=5$ (bilanciamento velocità/qualità)

Listing 15: Windows CMD

```
python src\eeg_ml_pipeline.py ^
  --mode real ^
  --input .\output ^
```

```
--output ./ml_results_kfold ^
--cv group_kfold ^
--n_splits 5
```

Listing 16: macOS / Linux Terminal

```
python3 src/eeg_ml_pipeline.py \
--mode real \
--input ./output \
--output ./ml_results_kfold \
--cv group_kfold \
--n_splits 5
```

GroupKFold con k=10 (maggiore stabilità delle stime)

Listing 17: Windows CMD

```
python src\eeg_ml_pipeline.py ^
--mode real ^
--input .\output ^
--output ./ml_results_kfold10 ^
--cv group_kfold ^
--n_splits 10
```

Listing 18: macOS / Linux Terminal

```
python3 src/eeg_ml_pipeline.py \
--mode real \
--input ./output \
--output ./ml_results_kfold10 \
--cv group_kfold \
--n_splits 10
```

GroupKFold con k=2 (verifica rapida caricamento)

Listing 19: Windows CMD

```
python src\eeg_ml_pipeline.py ^
--mode real ^
--input .\output ^
--output ./ml_results_test ^
--cv group_kfold ^
--n_splits 2
```

Listing 20: macOS / Linux Terminal

```
python3 src/eeg_ml_pipeline.py \
--mode real \
--input ./output \
--output ./ml_results_test \
--cv group_kfold \
--n_splits 2
```

8.3 Modalità con dati sintetici

Per testare la pipeline senza il dataset reale, è disponibile la modalità sintetica che genera dati EEG simulati con struttura realistica:

Listing 21: Windows CMD

```
python src\eeg_ml_pipeline.py ^
  --mode synthetic ^
  --n_subjects 10 ^
  --output .\ml_results_synthetic
```

Listing 22: macOS / Linux Terminal

```
python3 src/eeg_ml_pipeline.py \
  --mode synthetic \
  --n_subjects 10 \
  --output ./ml_results_synthetic
```

Nota – macOS: python vs python3

Su macOS, il comando `python` potrebbe puntare a Python 2 (sistemi meno recenti) oppure non essere disponibile. È consigliabile usare sempre `python3` per garantire l'uso di Python 3. In alternativa, verificare la versione attiva con: `python -version` e, se necessario, usare un virtual environment: `source venv/bin/activate`.

8.4 Riepilogo argomenti da riga di comando

Tabella 4: Argomenti accettati da `eeg_ml_pipeline.py`

Argomento	Tipo	Descrizione	Default
<code>-mode</code>	str	<code>real</code> o <code>synthetic</code>	<code>synthetic</code>
<code>-input</code>	str	Directory output preprocessing	<code>./output</code>
<code>-output</code>	str	Directory output risultati ML	<code>./ml_results</code>
<code>-cv</code>	str	<code>loso</code> o <code>group_kfold</code>	<code>loso</code>
<code>-n_splits</code>	int	k per GroupKFold (ignorato con <code>loso</code>)	5
<code>-n_jobs</code>	int	Processi paralleli (-1 = tutti i core)	-1
<code>-n_subjects</code>	int	Soggetti sintetici (solo <code>synthetic</code>)	10
<code>-no_save_models</code>	flag	Non salvare i modelli <code>.joblib</code>	disattivo

8.5 File di output prodotti

Tabella 5: File di output prodotti nella directory `-output`

File	Contenuto
<code>results_summary.csv</code>	Tabella media $\pm$ std per ogni metrica e modello
<code>feature_matrix.npy</code>	Matrice feature X shape (<code>n_epochs</code> , 105)
<code>feature_names.csv</code>	Nomi delle 105 feature (<code>Fz_theta</code> , <code>O1_alpha</code> , ...)
<code>feature_ranking.csv</code>	Ranking feature con importanza RF e coefficienti LR
<code>confusion_matrices.png</code>	CM normalizzata aggregata LOSO per i 3 modelli
<code>roc_curves.png</code>	ROC media $\pm$ std per fold, per ogni modello
<code>feature_importance.png</code>	Top-20 feature: importanza RF e coefficienti LR
<code>metrics_boxplot.png</code>	Distribuzione metriche per fold (boxplot)
<code>saved_models/*.joblib</code>	Modelli addestrati sull'intero dataset

9 Risultati Ottenuti e Confronto con la Letteratura

9.1 Statistiche del dataset preprocessato

La pipeline è stata eseguita su 55 soggetti (su 59 totali; 4 cartelle risultano mancanti nel dataset scaricato). I risultati sono riassunti nella tabella seguente:

Tabella 6: Statistiche aggregate del dataset per la pipeline ML

Parametro	Valore
Soggetti elaborati	55/59
Trial totali	$55 \times 200 = 11.000$
Epoche per soggetto	200 (100% dei trial, tutti validi)
Shape feature matrix X	(11.000, 105) – 21 canali $\times$ 5 bande
Trial advantageous (media)	≈ 101 (50.5%)
Trial disadvantageous (media)	≈ 99 (49.5%)
Fold LOSO	55 (uno per soggetto)
Campioni training per fold (media)	≈ 10.800 (54 soggetti $\times$ 200 trial)
Campioni test per fold	200 (1 soggetto)

9.2 Risultati LOSO (media $\pm$ std su 55 fold)

Tabella 7: Risultati della cross-validation LOSO per i tre modelli

Modello	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.521 ± 0.098	0.529 ± 0.179	0.520 ± 0.277	0.475 ± 0.188	0.545 ± 0.063
SVM (RBF)	0.520 ± 0.099	0.534 ± 0.165	0.451 ± 0.289	0.431 ± 0.199	0.553 ± 0.068
Random Forest	0.513 ± 0.084	0.529 ± 0.133	0.546 ± 0.259	0.492 ± 0.159	0.532 ± 0.059

Tutti e tre i modelli ottengono performance superiori al caso fortuito ($AUC > 0.5$) in modo consistente. Il SVM ottiene la ROC-AUC più elevata (0.553), mentre il Random Forest mostra il recall più alto (0.546). La Logistic Regression rappresenta il miglior compromesso tra performance e interpretabilità.

9.3 Confronto con la letteratura EEG-IGT

Il paper originale [1] non include benchmark di classificazione ML (è un data descriptor). Il confronto viene effettuato rispetto agli studi citati nel paper:

Tabella 8: Confronto con la letteratura EEG-IGT citata nel paper

Studio	Metodo	AUC	Note
Balconi & Angioletti (2022) [5]	SVM, feature spettrali, cross-subject	0.55–0.62	Setting analogo
Bianchin & Angrilli (2011) [3]	ERP (DPN), trial-by-trial	N/A	Feature ERP
Cui et al. (2013) [4]	ERP, incertezza	N/A	Theta Fz
Questa pipeline (2026)	PSD Welch, LOSO, LR/SVM/RF	0.532–0.553	55 soggetti

I risultati ottenuti (AUC 0.53–0.55) sono nel range inferiore di quello riportato da Balconi & Angioletti [5], coerentemente con la maggiore severità della LOSO pura e con l’uso di soli feature PSD senza feature temporali o di connettività.

10 Scelte Implementative e Motivazioni

10.1 PSD invece di analisi ERP

Il paper valida il dataset tramite analisi ERP (FRN, P300, N400 in Fig. 9). La pipeline usa PSD invece di feature ERP per la classificazione ML. La motivazione è statistico-metodologica: le feature ERP di singolo trial hanno SNR insufficiente senza averaging. Con ~ 100 trial per classe per soggetto, la stima ERP è instabile, producendo feature ad alta varianza che degradano la generalizzazione cross-subject. La PSD, stimata su una finestra di 2s, fornisce feature stabili e coerenti con i correlati neurali identificati dal paper.

10.2 Classificazione binaria (A/B vs C/D)

Il dataset registra la scelta per singolo deck (A, B, C, D). La pipeline aggrega A e B come classe 0 e C e D come classe 1. Questa scelta è motivata dalla teoria IGT di Bechara et al. [2]: la distinzione clinicamente rilevante è tra scelte immediate (A/B, alto guadagno immediato, perdita netta a lungo termine) e scelte ottimali a lungo termine (C/D, bilancio positivo). La Fig. 7 del paper usa esplicitamente questa dicotomia come metrica di performance.

10.3 Gestione sbilanciamento: `class_weight='balanced'`

Sebbene la distribuzione media sia quasi bilanciata (50.5% vs 49.5%), la distribuzione per singolo soggetto può essere significativamente sbilanciata (es. soggetto s-51: 167/33, soggetto s-36: 59/141). Il parametro `class_weight='balanced'` corregge automaticamente pesando ogni campione inversamente alla frequenza della propria classe:

$$w_i = \frac{n_{\text{tot}}}{2 \cdot n_{y_i}}, \quad y_i \in \{0, 1\}$$

prevenendo il bias del classificatore verso la classe maggioritaria nel fold.

10.4 Parallelizzazione con backend `loky`

La scelta del backend `loky` (process-based) invece di `threading` è motivata dal Global Interpreter Lock (GIL) di Python, che impedisce il vero parallelismo a thread per codice CPU-bound come il training ML. Il backend `loky`, implementato tramite processi separati, non soffre di questa limitazione ed è il backend raccomandato su Windows.

10.5 Finestra pre-decisionale $[-2\text{s}, 0\text{s}]$

La finestra eredita la scelta del preprocessing della Parte I, motivata da:

- **Decision Preceding Negativity (DPN)** [3]: negatività frontale in Fz che inizia ~ 500 ms prima della decisione
- **Slow Cortical Potentials (SCP)**: sviluppati su scale temporali di 1–2 secondi, richiedono una finestra ampia per essere catturati

- **Attività theta frontale:** massima nei 1.5–2 secondi che precedono la scelta nell'IGT [4]
- **Assenza di feedback:** la finestra non include il feedback ($t > 0$), evitando contaminazione con componenti post-decisionali come FRN e P300

11 Conclusioni e Sviluppi Futuri

11.1 Riepilogo

La pipeline di feature extraction e machine learning implementata elabora automaticamente le epoche EEG della Parte I, producendo risultati di classificazione cross-subject valutati con metodologia LOSO. I punti chiave sono:

1. **Feature extraction robusta:** PSD Welch su 5 bande $\times$ 21 canali = 105 feature per epoca, coerenti con i correlati neurali del paper.
2. **Tre classificatori complementari:** Logistic Regression (baseline lineare, interpretabile), SVM RBF (massima AUC), Random Forest (feature importance).
3. **Validazione cross-subject rigorosa:** LOSO garantisce assenza di data leakage e stima la generalizzazione a nuovi individui.
4. **Performance coerenti:** AUC 0.53–0.55, nel range atteso per classificazione trial-by-trial EEG-IGT con feature spettrali e LOSO.
5. **Parallelizzazione efficiente:** riduzione del tempo di esecuzione di 4–6 $\times$ su CPU multi-core tramite `joblib.Parallel`.
6. **Output completo:** CSV risultati, PNG visualizzazioni, modelli `.joblib` per deployment.

11.2 Sviluppi futuri (prossime fasi)

- **Feature di connettività:** coerenza theta Fz–Cz come proxy della comunicazione prefrontale-cingolata, asimmetria alpha frontale ($F4_\alpha - F3_\alpha$).
- **Feature time-frequency:** wavelet di Morlet e trasformata di Hilbert-Huang per dinamiche non stazionarie durante la deliberazione.
- **Deep learning:** CNN 1D sulle epoche raw, LSTM per sequenze di trial, EEGNet come baseline specifica per EEG.
- **Esclusione elettrodi di riferimento:** A1 e A2 (auricolari) compaiono tra le feature più importanti per RF; la loro esclusione dalla feature matrix va investigata sistematicamente.
- **Replicazione analisi ERP del paper:** implementazione averaging ERP per confronto diretto con Fig. 9 (FRN, P300, N400) tramite `mne.Evoked`.
- **Correzione notch filter:** verifica PSD a 60 Hz (standard messicano) e aggiornamento `NOTCH_FREQ = 60.0` se presente un picco residuo.

Riferimenti bibliografici

- [1] Chávez-Sánchez M., Torres-Ramos S., Román-Godínez I., Salido-Ruiz R.A. *Behavioral and electroencephalographic dataset simultaneously acquired during the Iowa Gambling Task*. Scientific Data, 13:359, 2026. DOI: [10.1038/s41597-026-06662-0](https://doi.org/10.1038/s41597-026-06662-0)
- [2] Bechara A., Damasio A.R., Damasio H., Anderson S.W. *Insensitivity to future consequences following damage to human prefrontal cortex*. Cognition, 50:7–15, 1994.
- [3] Bianchin M., Angrilli A. *Decision preceding negativity in the Iowa Gambling Task: an ERP study*. Brain and Cognition, 75:273–280, 2011.
- [4] Cui J.F., Chen Y.H., Wang Y., Shum D.H., Chan R.C. *Neural correlates of uncertain decision making: ERP evidence from the Iowa Gambling Task*. Frontiers in Human Neuroscience, 7:776, 2013.
- [5] Balconi M., Angioletti L. *Neurophysiology of gambling behavior and internet use vulnerability: a comparison between behavioral and EEG measures*. Clinical EEG and Neuroscience, 53:268–277, 2022.
- [6] Aram S. et al. *The Iowa gambling task: A review of the historical evolution, scientific basis, and use in functional neuroimaging*. Sage Open, 9(3), 2019.
- [7] Steingroever H. et al. *Performance of healthy participants on the Iowa Gambling Task*. Psychological Assessment, 25:180, 2013.
- [8] Gramfort A. et al. *MEG and EEG data analysis with MNE-Python*. Frontiers in Neuroscience, 7:267, 2013.
- [9] Pedregosa F. et al. *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12:2825–2830, 2011.
- [10] Nunez P.L. et al. *A theoretical and experimental study of high resolution EEG based on surface Laplacians and cortical imaging*. Electroencephalography and Clinical Neurophysiology, 90:40–57, 1994.